

Problem Session 7: Concurrency and Thread

Kiatiasak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Worawut Sereethai	67070501040
Chanya Poolketkij	67070501058

1 The Program

```

1  /*
2  * PS7 -- a race condition in a concert ticket-booking system.
3  *
4  * The hall has SEATS seats. Several booking threads sell tickets from one
5  * shared counter, seats_left, with no synchronisation. Each sale checks that a
6  * seat is free, then writes back one fewer. When two threads read the same
7  * value, both sell a ticket and both write the same result, so one decrement
8  * is lost and more tickets are sold than the hall holds.
9  *
10 * gcc -o ps7 ps7.c -lpthread
11 * ./ps7 1          one booking thread          (task 1)
12 * ./ps7 4          four threads, no forced context switch (task 2)
13 * ./ps7 4 yield    four threads, yielding mid-update (task 3)
14 */
15 #include <pthread.h>
16 #include <sched.h>
17 #include <stdio.h>
18 #include <stdlib.h>
19 #include <string.h>
20
21 #define SEATS      1000000
22 #define MAX_THREADS 16
23
24 /* The shared variable. volatile so that every access is a real load or store
25 * from memory, as the source reads, rather than a value the compiler keeps in
26 * a register across iterations. */
27 static volatile int seats_left = SEATS;
28
29 static int force_switch;    /* set by the "yield" argument */
30
31 typedef struct {
32     int id;
33     long sold;              /* private to one thread: needs no locking */
34 } Booker;
35
36 static void *book(void *arg)
37 {
38     Booker *me = arg;
39
40     for (;;) {
41         if (seats_left <= 0)          /* check: is a seat free? */
42             break;
43
44         if (force_switch) {
45             int reg = seats_left;    /* load memory into a register */
46             reg = reg - 1;            /* decrement the register */
47             sched_yield();            /* force a context switch */
48             seats_left = reg;        /* store the register back */
49         } else {
50             seats_left = seats_left - 1; /* act: take the seat */
51         }
52         me->sold++;
53     }
54     return NULL;
55 }

```

```

57 int main(int argc, char *argv[])
58 {
59     int nthreads = argc > 1 ? atoi(argv[1]) : 4;
60     force_switch = argc > 2 && strcmp(argv[2], "yield") == 0;
61
62     if (nthreads < 1 || nthreads > MAX_THREADS) {
63         fprintf(stderr, "usage: %s [threads 1-%d] [yield]\n", argv[0], MAX_THREADS);
64         return 1;
65     }
66
67     pthread_t tid[MAX_THREADS];
68     Booker    booker[MAX_THREADS];
69
70     printf("%d booking thread(s), %d seats, forced context switch: %s\n",
71           nthreads, SEATS, force_switch ? "yes" : "no");
72
73     for (int i = 0; i < nthreads; i++) {
74         booker[i].id = i + 1;
75         booker[i].sold = 0;
76         pthread_create(&tid[i], NULL, book, &booker[i]);
77     }
78
79     long total = 0;
80     for (int i = 0; i < nthreads; i++) {
81         pthread_join(tid[i], NULL);
82         printf("  thread %d sold %ld tickets\n", booker[i].id, booker[i].sold);
83         total += booker[i].sold;
84     }
85
86     printf("tickets sold: %ld for %d seats, seats left: %d\n",
87           total, SEATS, seats_left);
88     if (total == SEATS)
89         printf("result: correct\n");
90     else
91         printf("result: OVERSOLD by %ld tickets\n", total - SEATS);
92     return 0;
93 }

```

Listing 1: Source: ps7.c

The example is a concert box office rather than the lecture's shared counter. A hall of one million seats is sold by several booking threads that share one variable, `seats_left`. To sell a ticket a thread checks that a seat is free and then writes back one fewer; nothing makes that pair of steps indivisible. Each thread counts its own sales in a private variable, so the total sold is exact and any excess over one million is tickets sold for seats that do not exist.

The number of threads comes from the first argument. With a second argument, `yield`, the decrement is split into the explicit load, decrement, `sched_yield()` and store of the sheet's example, forcing a context switch between reading the counter and writing it back.

2 Task 1: One Thread

```
--- task 1, run 1 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 2 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 3 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 4 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 5 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 6 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct

--- task 1, run 7 ---
$ ./ps7 1
1 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1000000 tickets
tickets sold: 1000000 for 1000000 seats, seats left: 0
result: correct
```

Listing 2: Seven runs of `./ps7 1`

With a single thread the program is correct in every run: exactly one million tickets are sold and no seats are left. Only one thread ever reads or writes `seats_left`, so no update can interleave with another and the result is deterministic.

3 Task 2: Several Threads, No Forced Context Switch

```
--- task 2, run 1 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 970877 tickets
  thread 2 sold 756499 tickets
  thread 3 sold 942701 tickets
  thread 4 sold 670539 tickets
tickets sold: 3340616 for 1000000 seats, seats left: 0
result: OVERSOLD by 2340616 tickets

--- task 2, run 2 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 853406 tickets
  thread 2 sold 786425 tickets
  thread 3 sold 669075 tickets
  thread 4 sold 654655 tickets
tickets sold: 2963561 for 1000000 seats, seats left: -2
result: OVERSOLD by 1963561 tickets

--- task 2, run 3 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 485998 tickets
  thread 2 sold 553269 tickets
  thread 3 sold 506288 tickets
  thread 4 sold 915276 tickets
tickets sold: 2460831 for 1000000 seats, seats left: -1
result: OVERSOLD by 1460831 tickets

--- task 2, run 4 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 1214589 tickets
  thread 2 sold 781397 tickets
  thread 3 sold 582172 tickets
  thread 4 sold 778077 tickets
tickets sold: 3356235 for 1000000 seats, seats left: 0
result: OVERSOLD by 2356235 tickets

--- task 2, run 5 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 639517 tickets
  thread 2 sold 614481 tickets
  thread 3 sold 894386 tickets
  thread 4 sold 785616 tickets
tickets sold: 2934000 for 1000000 seats, seats left: 0
result: OVERSOLD by 1934000 tickets

--- task 2, run 6 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 843675 tickets
  thread 2 sold 908994 tickets
  thread 3 sold 580894 tickets
  thread 4 sold 605849 tickets
tickets sold: 2939412 for 1000000 seats, seats left: 0
result: OVERSOLD by 1939412 tickets

--- task 2, run 7 ---
$ ./ps7 4
4 booking thread(s), 1000000 seats, forced context switch: no
  thread 1 sold 984965 tickets
  thread 2 sold 618196 tickets
  thread 3 sold 671487 tickets
  thread 4 sold 611313 tickets
tickets sold: 2885961 for 1000000 seats, seats left: 0
result: OVERSOLD by 1885961 tickets
```

Listing 3: Seven runs of `./ps7 4`

Run	Tickets sold	Oversold by	Seats left
1	3,340,616	2,340,616	0
2	2,963,561	1,963,561	-2
3	2,460,831	1,460,831	-1
4	3,356,235	2,356,235	0
5	2,934,000	1,934,000	0
6	2,939,412	1,939,412	0
7	2,885,961	1,885,961	0

Table 1: Final totals of task 2

With four threads the outcome is both incorrect and non-deterministic across all runs: far more tickets are sold than the hall holds, and the counter can even end below zero. No yield is needed for this — the four threads run in parallel on separate cores and interleave on their own. The Discussion below explains both effects.

4 Task 3: Forced Context Switch

```
--- task 3, run 1 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 974612 tickets
  thread 2 sold 985241 tickets
  thread 3 sold 956641 tickets
  thread 4 sold 990127 tickets
tickets sold: 3906621 for 1000000 seats, seats left: 0
result: OVERSOLD by 2906621 tickets

--- task 3, run 2 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 994781 tickets
  thread 2 sold 993585 tickets
  thread 3 sold 997120 tickets
  thread 4 sold 991734 tickets
tickets sold: 3977220 for 1000000 seats, seats left: 0
result: OVERSOLD by 2977220 tickets

--- task 3, run 3 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 978872 tickets
  thread 2 sold 985593 tickets
  thread 3 sold 1011897 tickets
  thread 4 sold 986278 tickets
tickets sold: 3962640 for 1000000 seats, seats left: 0
result: OVERSOLD by 2962640 tickets

--- task 3, run 4 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 954999 tickets
  thread 2 sold 966836 tickets
  thread 3 sold 983761 tickets
  thread 4 sold 948853 tickets
tickets sold: 3854449 for 1000000 seats, seats left: 0
result: OVERSOLD by 2854449 tickets

--- task 3, run 5 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 952878 tickets
  thread 2 sold 1012297 tickets
  thread 3 sold 981335 tickets
  thread 4 sold 984938 tickets
tickets sold: 3931448 for 1000000 seats, seats left: 0
result: OVERSOLD by 2931448 tickets

--- task 3, run 6 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 957783 tickets
  thread 2 sold 988187 tickets
  thread 3 sold 955115 tickets
  thread 4 sold 972485 tickets
tickets sold: 3873570 for 1000000 seats, seats left: 0
result: OVERSOLD by 2873570 tickets

--- task 3, run 7 ---
$ ./ps7 4 yield
4 booking thread(s), 1000000 seats, forced context switch: yes
  thread 1 sold 985269 tickets
  thread 2 sold 972796 tickets
  thread 3 sold 966003 tickets
  thread 4 sold 967716 tickets
tickets sold: 3891784 for 1000000 seats, seats left: 0
result: OVERSOLD by 2891784 tickets
```

Listing 4: Seven runs of `./ps7 4 yield`

Run	Tickets sold	Oversold by	Seats left
1	3,906,621	2,906,621	0
2	3,977,220	2,977,220	0
3	3,962,640	2,962,640	0
4	3,854,449	2,854,449	0
5	3,931,448	2,931,448	0
6	3,873,570	2,873,570	0
7	3,891,784	2,891,784	0

Table 2: Final totals of task 3

Forcing a context switch makes the overselling larger on average, and the totals still differ from run to run. A thread now holds its private copy of the counter across the `sched_yield()` while the other threads keep selling. When it resumes it stores its stale value, putting back every seat they sold in the meantime, so those seats are sold again.

5 Discussion

The cause is that selling a seat is a *read-modify-write* of shared state that is not atomic. `seats_left = seats_left - 1` is one line of C but three machine steps — load into a register, subtract, store — and each thread has its own registers. Whenever another thread’s store lands between one thread’s load and its store, that thread writes back a value computed from stale data and the other thread’s update is lost, so a seat is sold twice.

The check `seats_left <= 0` adds a second race, a *check-then-act* or time-of-check to time-of-use (TOCTOU) race: the decision is made on a value that may already be out of date when the decrement happens. Two threads can both see the last seat free and both take it, which is how the counter ends below zero.

Which interleavings occur depends on how the scheduler and the cores happen to run the threads, which is why the outcome differs from run to run while the single-threaded run is always correct. Removing the race requires making the check and the decrement one indivisible step, for example by holding a `pthread_mutex_t` around them.